Lab 5 Report

Feature detection, description, matching and tracking

Team Fullcans

Report date: 23 October 2026

Abstract

Matching two unrelated views and tracking features between consecutive video frames call for different pipelines. We built both kinds in the Lab 5 stencil (ROS Noetic, OpenCV 3.4.2 with contrib): SIFT, SURF, ORB and FAST+BRIEF trackers with kNN matching, Lowe's ratio test and RANSAC; a Harris + pyramidal Lucas–Kanade tracker; and Farneback dense optical flow. On the `box` image pair, where the box is rotated and about half as large in the scene, scale-invariant descriptors win: SURF finds the most inliers (88), SIFT reproduces the handout (603/969 keypoints, 79 inliers, 79 % inlier ratio), and ORB matches well once its scale pyramid is enabled. On the two video datasets every method keeps more than 84 % of its matches as inliers, and Harris+LK leads on both reliability (95.5–99.7 % inliers) and speed (3.0–5.9 ms per frame pair). Among the descriptor methods, ORB gives the best trade-off for onboard visual odometry: 98.1 % inliers on the real data at 9–12 ms per pair, against 64–145 ms for SIFT.

1 Individual work

Throughout this section the camera has principal point $(0, 0)$, focal length 1, $s_x = s_y = 1$ and $s_\theta = 0$, so a point $p = [x\ y\ z]^\top$ with $z > 0$ projects to $(u, v) = (x/z,\ y/z)$, and the pixel (u, v) corresponds to the ray of points λw, $\lambda > 0$, with $w = [u\ v\ 1]^\top$.

1.1 Deliverable 1: Perspective projection of a sphere

1. Sphere on the optical axis. Let the centre be $c = [0\ 0\ d]^\top$. Because $d > r + 1$, every point of the sphere has $z \geq d - r > 1$, so the sphere lies entirely in front of the image plane $z = 1$ and is fully visible. A pixel (u, v) belongs to the image of the sphere iff its ray $\{\lambda w\}$ meets the sphere, i.e. iff the distance from c to the line through the origin with direction w is at most r:

$$\frac{\|c \times w\|^2}{\|w\|^2} \leq r^2. \tag{1}$$

The boundary of the image (the *occluding contour*) is formed by the rays tangent to the sphere, where equality holds. With $c \times w = [-dv\ \ du\ \ 0]^\top$, $\|c \times w\|^2 = d^2(u^2 + v^2)$ and $\|w\|^2 = u^2 + v^2 + 1$, hence

$$d^2(u^2 + v^2) = r^2(u^2 + v^2 + 1) \quad \Longleftrightarrow \quad \boxed{u^2 + v^2 = \frac{r^2}{d^2 - r^2}}. \tag{2}$$

The projection is a disc centred at the principal point, bounded by a circle of radius $\rho = r/\sqrt{d^2 - r^2}$ (well defined since $d > r$). Geometrically, the tangent rays form a right circular cone with apex at the optical centre, axis along the optical axis and half-angle α with $\sin\alpha = r/d$. The image plane is perpendicular to the axis and cuts the cone in a circle of radius $\tan\alpha = r/\sqrt{d^2 - r^2}$. For $d \gg r$ this tends to r/d, the familiar "size divided by depth" rule.

2. Sphere at an arbitrary position. For a general centre $c = [c_x \; c_y \; c_z]^\top$, the identity $\|c \times w\|^2 = \|c\|^2\|w\|^2 - (c^\top w)^2$ turns the tangency condition $\|c \times w\|^2 = r^2\|w\|^2$ into

$$(c_x u + c_y v + c_z)^2 = k\,(u^2 + v^2 + 1), \qquad k = \|c\|^2 - r^2. \tag{3}$$

This equation is quadratic in (u, v), so the contour is always a **conic**. It is the intersection of the tangent cone (still a right circular cone, now with its axis along c) with an image plane that is no longer perpendicular to that axis. Rotating the image axes so that u' points along (c_x, c_y), writing $\rho_c = \sqrt{c_x^2 + c_y^2}$ and completing the square turns (3) into

$$(c_z^2 - r^2)\Big(u' - \frac{\rho_c c_z}{c_z^2 - r^2}\Big)^2 + k\,v'^2 = \frac{r^2 k}{c_z^2 - r^2}. \tag{4}$$

- If the sphere is fully in front of the camera ($c_z > r$, hence $k > 0$), both coefficients are positive and the projection is an **ellipse** centred at $u'_0 = \rho_c c_z/(c_z^2 - r^2)$, $v'_0 = 0$, with semi-axes

$$a_{\text{radial}} = \frac{r\sqrt{\|c\|^2 - r^2}}{c_z^2 - r^2}, \qquad a_\perp = \frac{r}{\sqrt{c_z^2 - r^2}}.$$

  The major axis points radially away from the principal point, and the elongation $a_{\text{radial}}/a_\perp = \sqrt{(\|c\|^2 - r^2)/(c_z^2 - r^2)} \geq 1$ grows as the sphere moves towards the border of the field of view. The centre of the ellipse is shifted outwards from ρ_c/c_z, the projection of the sphere centre. For $c_x = c_y = 0$ we recover the circle of part 1.

- If the sphere touches the plane $z = 0$ through the optical centre ($c_z = r$), the u'^2 coefficient vanishes and the contour is a **parabola**. If it crosses that plane ($|c_z| < r$, camera still outside the sphere), the coefficients have opposite signs and the contour is (one branch of) a **hyperbola**. In both cases part of the sphere is behind the camera.

Figure 1 checks these formulas numerically. We project 2×10^5 points sampled on a unit sphere at depth $c_z = 5$; for $c_x = 0, \ldots, 10$ the extents of the projected points match the analytic ellipse to within 4×10^{-6}. The ellipse stretches radially as the sphere moves off axis ($a_{\text{radial}}/a_\perp = 2.27$ at 63°), and its centre lies further out than the projected sphere centre.

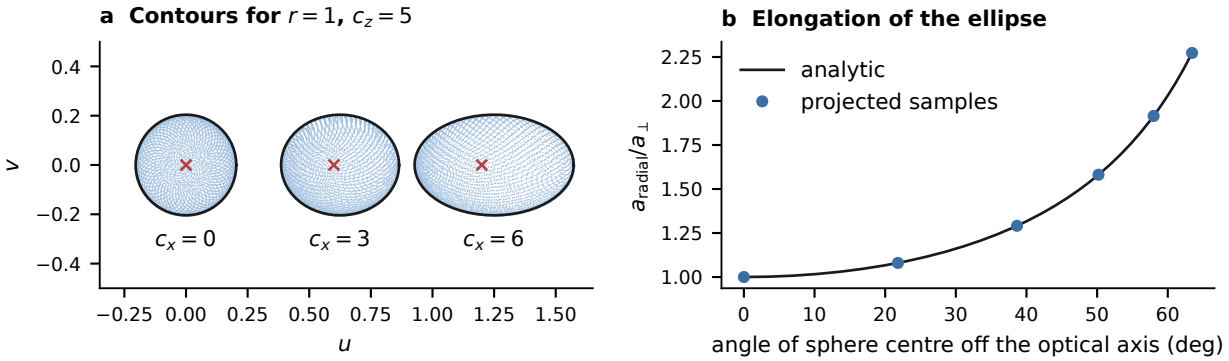


Figure 1: Perspective projection of a sphere ($r = 1$, $c_z = 5$). (a) Projected surface samples (blue), analytic contour from (3) (black) and projection of the sphere centre (red cross) for $c_x = 0, 3, 6$. (b) Elongation $a_{\text{radial}}/a_\perp$ against the angle of the sphere centre off the optical axis: analytic (line) and measured from the projected samples (dots).

1.2 Deliverable 2: Vanishing points

1. Vanishing point of a family of parallel lines. Write a 3D line as $p(\lambda) = p_0 + \lambda u$ with $\|u\| = 1$. Its image is

$$\pi(p(\lambda)) = \left(\frac{p_{0,x} + \lambda u_x}{p_{0,z} + \lambda u_z},\ \frac{p_{0,y} + \lambda u_y}{p_{0,z} + \lambda u_z}\right). \tag{5}$$

If $u_z \neq 0$, letting $\lambda \to \infty$ in the direction in which the line moves away from the camera ($\lambda u_z > 0$) gives

$$\boxed{v_\infty = \left(\frac{u_x}{u_z},\ \frac{u_y}{u_z}\right)}, \tag{6}$$

which depends only on the direction u and not on the point p_0. Every line with direction u, in particular both parallel lines $p_0 + \lambda u$ and $q_0 + \mu u$, therefore passes through v_∞ in the image: v_∞ is the vanishing point. It is the image of the point at infinity $[u^\top\ 0]^\top$, i.e. the intersection of the image plane with the ray through the optical centre parallel to the lines. With a general calibration matrix K it reads $v_\infty \simeq Ku$ in homogeneous coordinates.

Geometric view. Each 3D line spans, together with the optical centre O, a plane whose intersection with the image plane is the image line. For two parallel lines both planes contain the direction u, so they intersect along the line through O with direction u, which hits the image plane exactly at v_∞.

2. When do parallel lines stay parallel? Claim: the images of two distinct parallel 3D lines (neither through O) are parallel if and only if their direction is parallel to the image plane, $u_z = 0$.

Proof. ($\Leftarrow$) If $u_z = 0$ the depth along the line is constant, $z = p_{0,z}$, and $\pi(p(\lambda)) = \frac{1}{p_{0,z}}[p_{0,x}\ p_{0,y}]^\top + \frac{\lambda}{p_{0,z}}[u_x\ u_y]^\top$: the image is a line with direction $[u_x\ u_y]^\top$. The second line has the same image direction, so the two image lines are parallel (the vanishing point Ku has third coordinate 0 and lies at infinity). ($\Rightarrow$) If $u_z \neq 0$, both image lines contain the finite point v_∞ from part 1. They intersect there and cannot be parallel; they can only coincide, when the two 3D lines are coplanar with O. □

In the geometric picture, the two planes through O meet in the line through O with direction u, and this line meets the image plane at a finite point unless it is parallel to it ($u_z = 0$). For a camera looking straight at a wall, for example, the horizontal and vertical edges on the wall stay parallel in the image, while edges receding in depth converge. Figure 2 projects five parallel lines with different starting points for both cases.

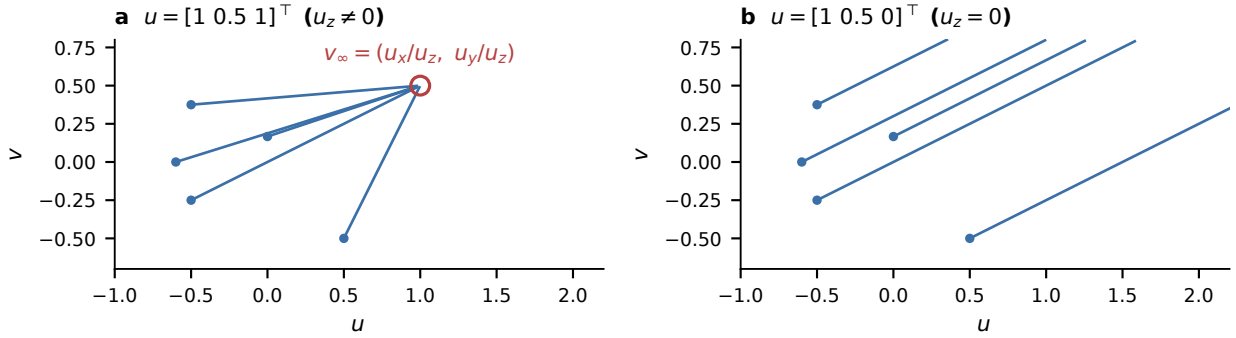


Figure 2: Images of five parallel 3D lines (dots: projected starting points p_0). (a) With $u_z \neq 0$ all images converge to $v_\infty = (1, 0.5)$. (b) With $u_z = 0$ they keep the image direction $[u_x\ u_y]^\top$ and stay parallel.

2 Team work

2.1 Implementation and setup

Build. The official 2023 ROS1 stencil (`feature_tracking`, package `lab_5`) is built in a catkin workspace together with `opencv3_catkin` (OpenCV 3.4.2 + contrib, whose `xfeatures2d` module still contains the patented SIFT and SURF). Relative to the official package we switched off modules the lab does not use (Python bindings, VTK/viz, FFMPEG) and use the system glog instead of `glog_catkin`. Because `cv_bridge` exports the system OpenCV 4.2 libraries, we also filter them out of the link line of `lab_5`; otherwise two `highgui` versions (GTK2 and GTK3) are loaded and `imshow` aborts. `scripts/build_workspace.sh` builds everything (OpenCV takes 4 min on 12 cores).

Descriptor-based trackers (D3–D6). We compare SIFT [1], SURF [2], ORB [3] and FAST [4] + BRIEF [5]. `FeatureTracker::trackFeatures` detects keypoints in both images (`detector->detect`), describes them (`detector->compute`; FAST uses a separate `BriefDescriptorExtractor`), and calls `matchDescriptors`. That function runs `knnMatch` with $k = 2$ and keeps `m[0]` if `m[0].distance < 0.8 m[1].distance`; queries with fewer than two neighbours are dropped. SIFT and SURF use the default `FlannBasedMatcher` (randomised kd-trees on float descriptors [6]). ORB and BRIEF descriptors are binary and are matched with a brute-force Hamming matcher (`BFMatcher(NORM_HAMMING)`); a parameter switches to a FLANN LSH index, compared below. The good matches go to the provided `inlierMaskComputation` (fundamental matrix with RANSAC, 3 px epipolar threshold), and two `cv::drawMatches` calls (`DRAW_OVER_OUTIMG`) draw the inliers in green over the red outliers. The provided code averages all statistics, and we also log them per frame to CSV.

Two fixes to the stencil. In video mode, `imageCallback` stored `prev_image = image`, where `image` comes from `cv_bridge::toCvShare` and does not own its pixels; we keep a `clone()` instead. The stencil also matched the first frame with itself (100 % inliers); we now skip it.

Statistics. As in the handout table, "# of Matches" is the number of kNN queries (one per keypoint of image 1), "Good Matches" are those that pass the ratio test, and the inlier ratio is #inliers/#good matches. We add the wall-clock time of detection, description and matching for one image pair (both images processed, drawing excluded; Intel Core i5-12400F, Release build).

2.2 Deliverable 3: SIFT keypoints and descriptors

Figure 3 shows the 603 and 969 SIFT keypoints detected in `box.png` and `box_in_scene.png`, drawn with their scale (circle radius) and orientation.

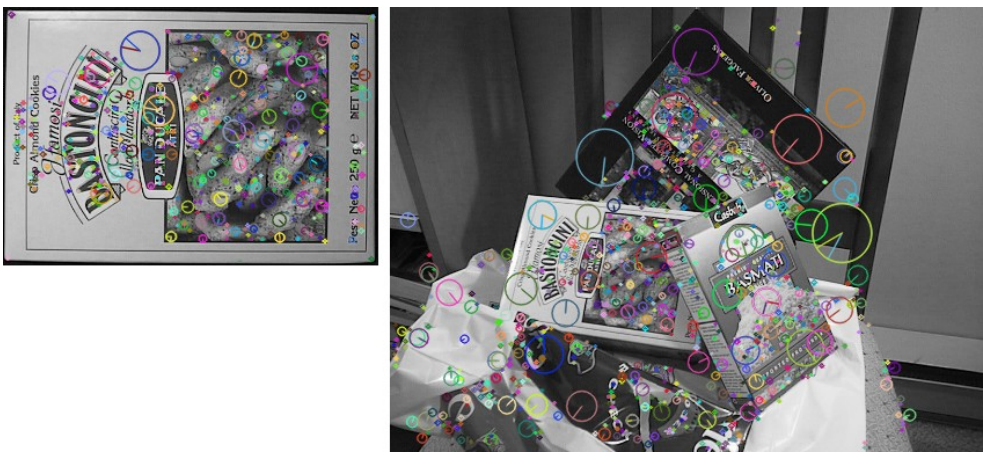


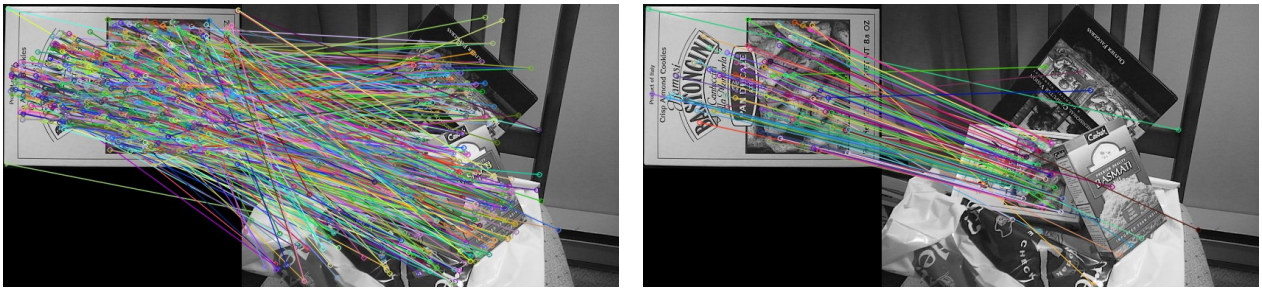
Figure 3: SIFT keypoints (`DRAW_RICH_KEYPOINTS`): 603 in `box.png` (left), 969 in `box_in_scene.png` (right).

2.3 Deliverable 4: Descriptor-based matching

1. The SIFT descriptor has **128 dimensions** (a 4×4 grid of 8-bin gradient orientation histograms), stored as `CV_32F`; the descriptor matrices are 603×128 and 969×128.

2. Figure 4(a) shows the best FLANN match of every keypoint of `box.png` (603 matches). Most of them are wrong, because every keypoint gets a nearest neighbour, including those on the background of `box.png` that have no counterpart in the second image.

3–4. The ratio test with threshold 0.8 keeps 98 matches (Fig. 4(b)), and almost all of them connect the box to its copy in the scene. FLANN's kd-trees are randomised, so over 20 runs the count ranges from 95 to 104 (mean 99.7; the handout reports 93).



(a) All 603 nearest-neighbour matches. **(b)** 98 matches after the ratio test.

Figure 4: SIFT + FLANN matches from `box.png` to `box_in_scene.png`.

2.4 Deliverable 5: Matching quality

Figure 5 shows the inliers (green) and outliers (red) among the 98 good matches of that run.

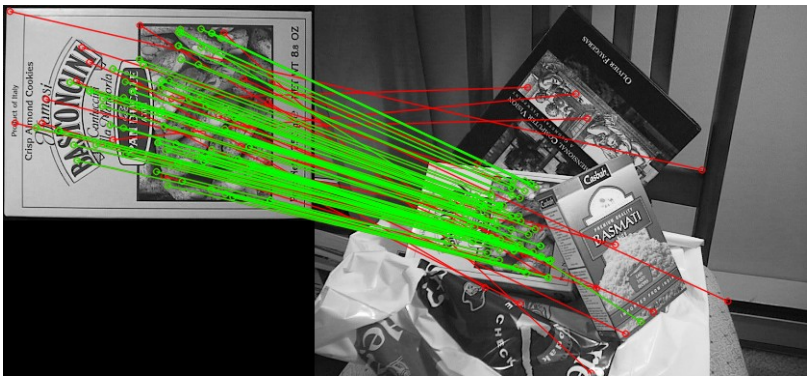


Figure 5: SIFT: 76 RANSAC inliers (green) and 22 outliers (red) among the 98 ratio-test matches.

Table 1 (first column) gives the statistics averaged over 20 runs, close to the handout values (603 / 969 / 603 / 93 / 78 / 83.9 %).

Table 1: Statistics on `box.png`→`box_in_scene.png`, mean of 20 runs. ORB: 2000 features, 8 pyramid levels; FAST: threshold 10, 32-byte BRIEF; others default.

Statistics	SIFT	SURF	ORB	FAST+BRIEF
# of Keypoints in Img 1	603	997	1589	2266
# of Keypoints in Img 2	969	1822	1999	3653
# of Matches	603	997	1589	2266
# of Good Matches	100	137	73	41
# of Inliers	79	88	55	17
Inlier Ratio	79.0%	64.2%	75.3%	41.5%
Time / pair [ms]	64.8	58.4	19.4	25.0

2.5 Deliverable 6.a: Comparison on the image pair

The box appears rotated and about 2× smaller in the scene, so this pair tests scale and rotation invariance. SURF finds the most inliers and SIFT the highest inlier ratio.

Figure 6 shows where each pipeline loses its matches. The ratio test keeps 17 % of the SIFT and 14 % of the SURF queries, but only 5 % for ORB and 2 % for FAST+BRIEF; on this pair the descriptor, not RANSAC, sets the final count.

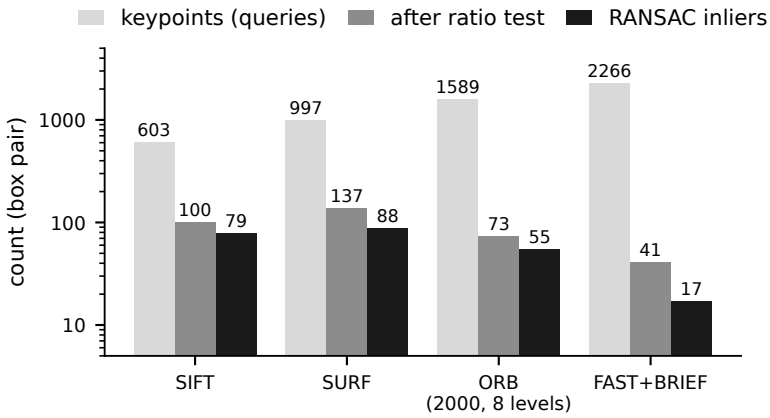


Figure 6: Matches surviving each stage on the box pair, log scale: kNN queries (one per keypoint of `box.png`), ratio-test matches and RANSAC inliers. Configurations as in Table 1.

Table 2 lists every configuration we ran.

Table 2: All configurations run on the image pair.

Method	Parameters	KP 1	Good	Inliers	Ratio [%]	ms
SIFT	default	603	100	79	79.0	64.8
SURF	Hessian 100 (default)	997	137	88	64.2	58.4
SURF	Hessian 400	786	119	76	63.4	43.2
ORB	500 feat., 1 level, BF (stencil)	500	13	7	53.8	4.3
ORB	500 feat., 1 level, FLANN-LSH	500	13	7	53.8	20.7
ORB	500 feat., 8 levels, BF	453	29	26	89.7	8.8
ORB	2000 feat., 8 levels, BF	1589	73	55	75.3	19.4
FAST+BRIEF	threshold 10, BF	2266	41	17	41.5	25.0
FAST+BRIEF	threshold 10, FLANN-LSH	2266	41	17	41.5	476.6
FAST+BRIEF	threshold 30, BF	1095	12	7	58.3	9.0

Four parameter choices explain the differences.

- **ORB needs its scale pyramid.** The stencil's `ORB::create(500, 1.2, 1)` uses one pyramid level to avoid the "Unknown interpolation method" error mentioned in the handout, which makes ORB single-scale and leaves 7 inliers. OpenCV 3.4.2 runs the 8-level pyramid without that error and gives 26 inliers (89.7 % ratio); with 2000 features ORB reaches 55 inliers at 19 ms, a third of the SIFT and SURF time.
- **The default SURF Hessian threshold (100) gives the most inliers.** Raising it to 400 saves 25 % of the time but drops the inliers from 88 to 76.

- **FAST+BRIEF is built for small viewpoint changes.** Neither FAST nor BRIEF is scale invariant and BRIEF is not rotation invariant, so this pair leaves it 41 good matches and 17 inliers; raising the FAST threshold from 10 to 30 removes good matches faster than corners. With so few correct matches, RANSAC also accepts some wrong ones (Fig. 7), because the fundamental-matrix test only checks the distance to an epipolar line.

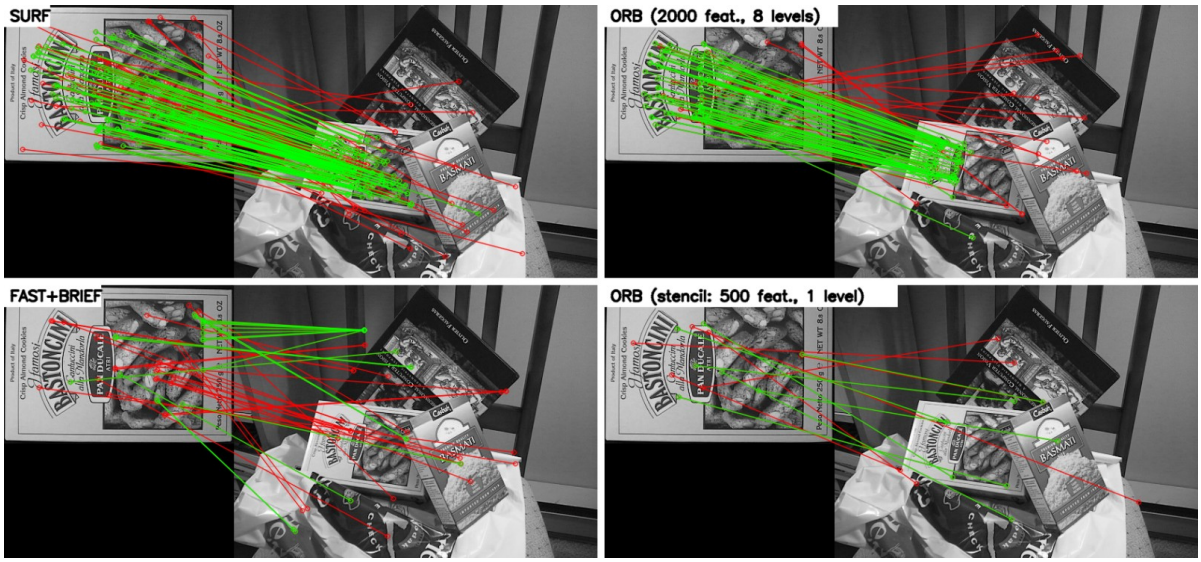


Figure 7: Inliers (green) and outliers (red) for SURF, ORB (2000 features, 8 levels), FAST+BRIEF and the stencil ORB (500 features, 1 level).

- **Brute-force Hamming beats FLANN-LSH for binary descriptors.** With an LSH index (`LshIndexParams(20, 10, 2)`), `FlannBasedMatcher` returns the same good matches as brute force (an independent Python check found the exact nearest neighbour for 498 of 500 ORB descriptors) but runs 5–20× slower, because the LSH index is rebuilt for every image pair and the descriptor sets are small. Brute-force Hamming distance is a XOR and a popcount per pair.

2.6 Deliverable 6.b: Real datasets

We ran every method on the two required bags and matched each frame against the previous one (current frame = image 1, previous frame = image 2). The bags play at real-time rate, and the subscriber queue (1000) is large enough that no frame is dropped: 1578–1583 and 1382–1385 frame pairs were processed, the few missing ones at the start and end of the bag while the node connects or shuts down. For every method we ran 2–3 parameter sets; the tables below report the set with the most inliers, which was the same on both datasets.

Table 3 gives the averages for `30fps_424x240_2018-10-01-18-35-06.bag`: real RealSense colour images, 424 × 240, 1584 frames of a hand-held walk through a tunnel.

Table 3: Average over all frame pairs of `30fps_424x240` (real data).

Statistics	SIFT	SURF	ORB	FAST+BRIEF	Harris+LK
# of Keypoints in Img 1	741	467	923	828	561
# of Keypoints in Img 2	741	467	922	828	561
# of Matches	741	467	923	828	561
# of Good Matches	412	334	680	635	554
# of Inliers	395	320	668	608	552
Inlier Ratio	94.6%	95.3%	98.1%	95.1%	99.7%
Time / pair [ms]	63.7	28.7	8.8	5.4	3.0

Table 4 gives the averages for `vnav-lab5-smooth-trajectory.bag`: simulated TESSE camera images, 720 × 480, 1386 frames.

Table 4: Average over all frame pairs of `vnav-lab5-smooth-trajectory` (simulated data).

Statistics	SIFT	SURF	ORB	FAST+BRIEF	Harris+LK
# of Keypoints in Img 1	270	1194	787	568	276
# of Keypoints in Img 2	270	1192	786	568	276
# of Matches	270	1194	787	568	276
# of Good Matches	169	716	435	392	269
# of Inliers	142	610	392	357	255
Inlier Ratio	84.4%	85.4%	89.9%	90.6%	95.5%
Time / pair [ms]	144.9	72.1	12.1	4.4	5.9

Parameters: SIFT contrast threshold 0.02 (default 0.04); SURF Hessian 100; ORB 2000 features, 8 levels; FAST threshold 10 + 32-byte BRIEF; ORB/BRIEF matched with brute-force Hamming; ratio 0.8 everywhere. Harris+LK: up to 1000 corners, quality level 0.01, minimum distance 5 px (Section 2.7). Table 5 lists all configurations we ran.

Table 5: All configurations run on the videos (bold: used in Tables 3–4).

		`30fps_424x240`				`smooth-trajectory`			
Method	Parameters	KP	Inl.	%	ms	KP	Inl.	%	ms
SIFT	contrast 0.04 (default)	404	249	95.7	64.1	148	81	86.4	131.8
SIFT	**contrast 0.02**	741	395	94.6	63.7	270	142	84.4	144.9
SURF	**Hessian 100**	467	320	95.3	28.7	1194	610	85.4	72.1
SURF	Hessian 400	266	200	97.0	16.4	476	260	87.3	41.2
ORB	500, 1 level (stencil)	316	227	97.9	2.6	279	152	90.7	3.1
ORB	1000, 8 levels	725	529	97.9	7.7	619	316	90.2	10.9
ORB	**2000, 8 levels**	923	668	98.1	8.8	787	392	89.9	12.1
FAST+BRIEF	**threshold 10**	828	608	95.1	5.4	568	357	90.6	4.4
FAST+BRIEF	threshold 20	342	274	96.6	2.4	300	195	92.1	2.8
FAST+BRIEF	threshold 30	193	158	96.9	1.6	181	120	92.0	2.3
Harris+LK	500 corners, 10 px	329	322	99.4	2.6	153	140	94.8	5.4
Harris+LK	**1000 corners, 5 px**	561	552	99.7	3.0	276	255	95.5	5.9

Figure 8 shows the inliers and outliers of the four descriptor methods on one frame pair of each dataset.

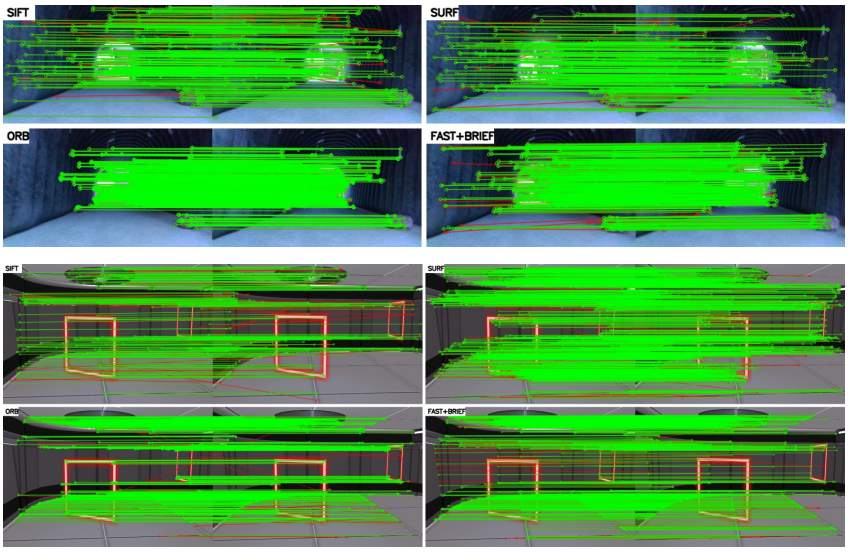


Figure 8: Inliers (green) and outliers (red) at frame pair 150 of each dataset (top: real, bottom: simulated). Full SIFT videos: `video/d6_sift_tracking_*.mp4`.

Conclusions.

- **Consecutive frames are much easier than the image pair.** Between two video frames the image moves by a few pixels with almost no scale or rotation change, so all methods reach 84–98 % inlier ratio, including FAST+BRIEF and single-level ORB. Invariance to large scale and rotation changes (SIFT, SURF, multi-scale ORB) pays off when matching unrelated views, as in loop closure, more than in frame-to-frame tracking.
- **Keypoint counts follow the thresholds and the scene.** SURF finds by far the most keypoints on the simulated images (1194) and SIFT the fewest (270, with the contrast threshold already halved); on the real images FAST and ORB find the most. Lowering a threshold raises the number of inliers almost proportionally at a small cost in inlier ratio.
- **Binary descriptors keep more of their matches.** On the real dataset FAST+BRIEF keeps 77 % and ORB 74 % of the kNN matches after the ratio test, against 56 % for SIFT, and ORB has the highest inlier ratio of the descriptor methods (98.1 %). On the simulated dataset all ratios are lower (84–91 %): large textureless walls and repeated structures (identical gates, floor tiles) give ambiguous descriptors, and Fig. 9 shows the ratio dropping when the camera faces such regions or turns quickly.

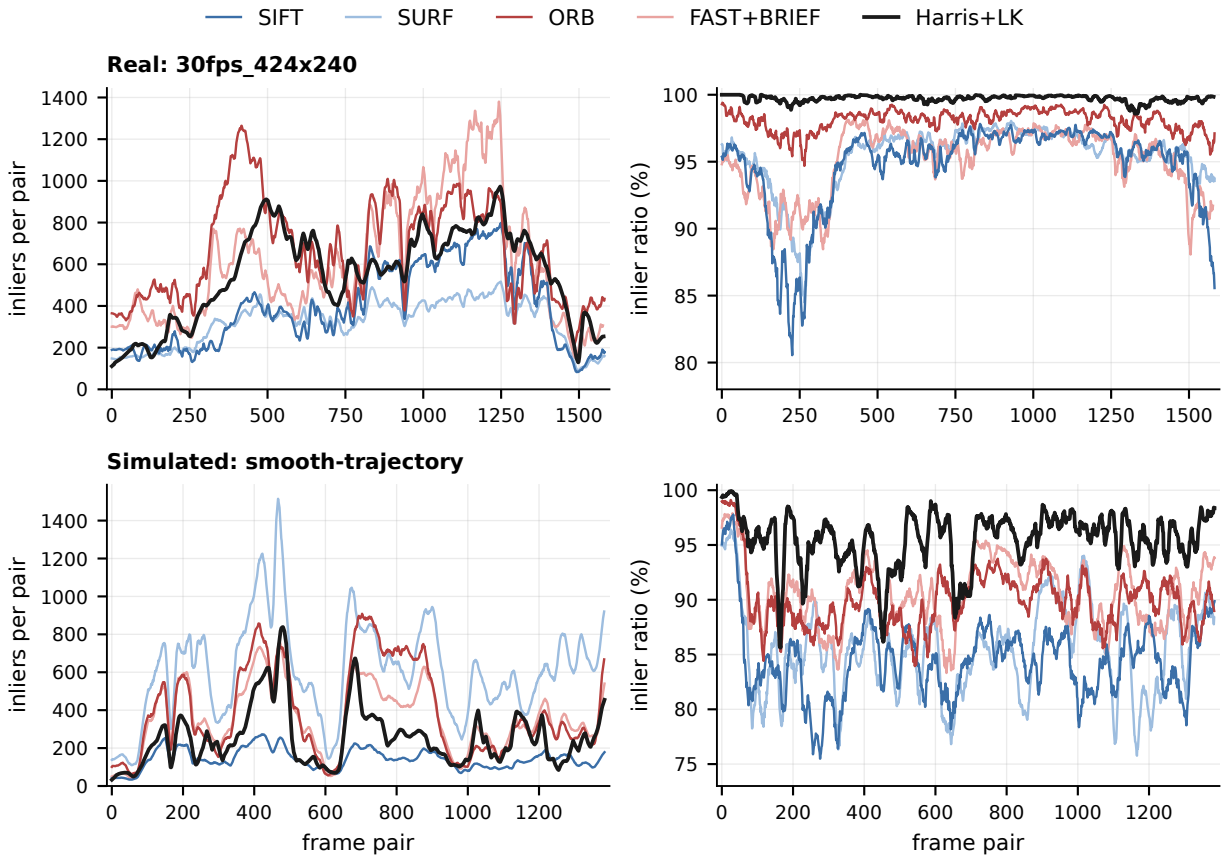


Figure 9: Inliers per frame pair (left) and inlier ratio (right) on the real (top) and simulated (bottom) dataset, smoothed with a centred 15-pair moving average (0.5 s). Note the truncated ratio axes. Harris+LK (black) has the highest ratio throughout; the descriptor methods dip when the camera turns quickly or faces textureless or repetitive regions.

- **Binary methods are an order of magnitude faster** (Fig. 10): FAST+BRIEF takes 4–5 ms and ORB 9–12 ms per pair, against 29–72 ms for SURF and 64–145 ms for SIFT. Each pair processes both images, so caching the previous frame's keypoints would roughly halve the detection cost. SIFT on the 720 × 480 simulated images exceeds the 33 ms budget of a 30 Hz camera four times over. Binary descriptors are also 8–16× smaller (32 bytes against 256/512 bytes for SURF/SIFT) and are compared with a Hamming distance instead of an L2 distance.

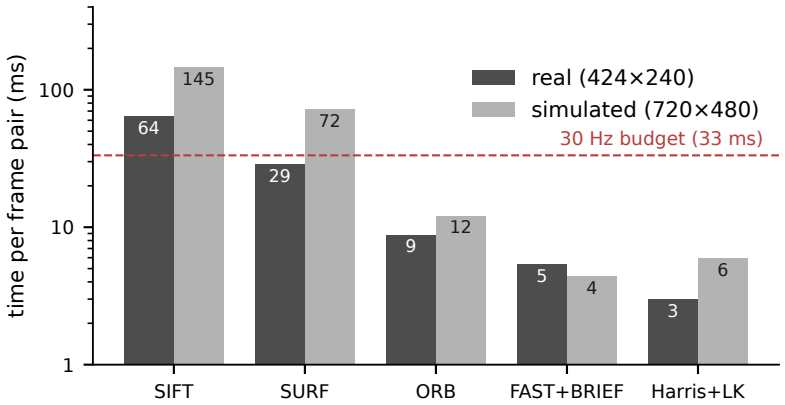


Figure 10: Average time per frame pair (detection and description of both frames plus matching; for LK, tracking plus re-detection), log scale. Only ORB, FAST+BRIEF and Harris+LK fit the 33 ms budget of a 30 Hz camera on both datasets. Timings varied by up to ±30 % between repeated runs (system load); counts by less than 1 %.

- **ORB suits onboard VO, SIFT/SURF suit offline matching.** For real-time VO/SLAM on a drone, with a high frame rate and limited compute, ORB is fast and invariant enough to rotation and scale for re-localisation, which is also the choice made in ORB-SLAM [7]. FAST+BRIEF is the fastest descriptor-based option when the inter-frame motion is small. SIFT and SURF handle the largest viewpoint and scale changes (box pair) and fit offline tasks such as structure from motion, image stitching or object recognition, where accuracy matters more than speed. Figure 11 places all configurations of Table 5 on a plot of time against inliers: within the 30 Hz budget, ORB gives the most inliers on both datasets (668 and 392 per pair).

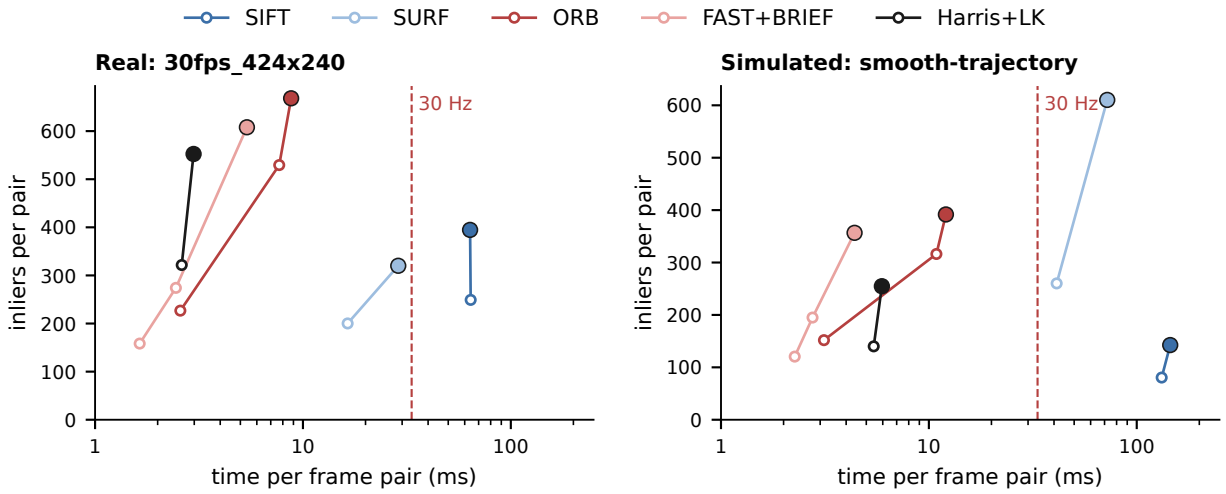
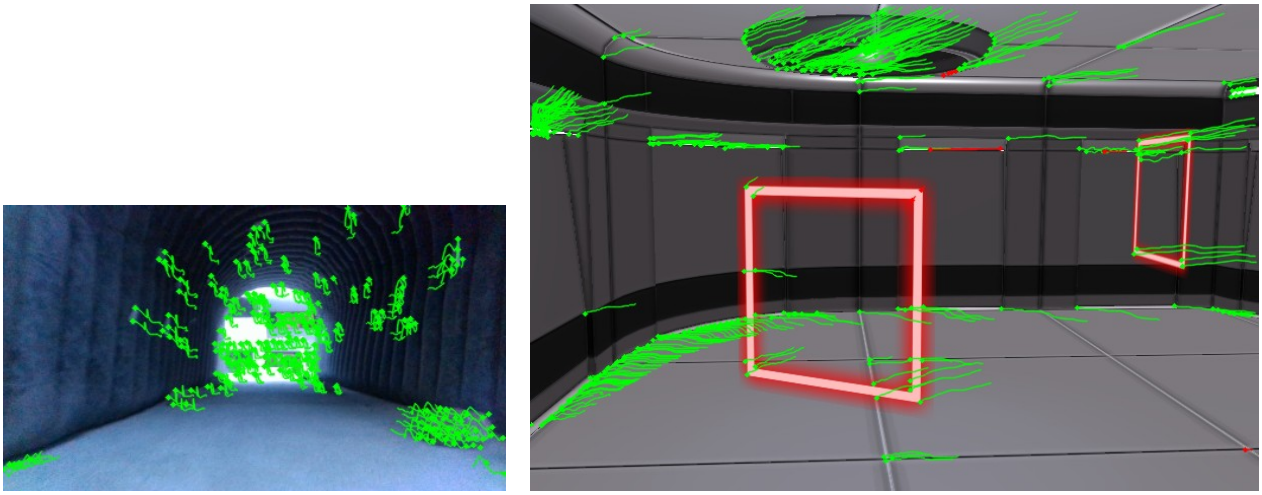


Figure 11: Time against inliers per frame pair for every configuration of Table 5. Lines join the configurations of one method in order of keypoint count; filled markers are the configurations used in Tables 3–4. Dashed line: 33 ms budget of a 30 Hz camera.

2.7 Deliverable 7: Harris corners + Lucas–Kanade tracker

Implementation. We combine Harris corners [8] with the Lucas–Kanade tracker [9]. Our `trackFeatures` converts the frame to grayscale and runs `calcOpticalFlowPyrLK` (21×21 window, 4 pyramid levels, at most 30 iterations / $\varepsilon = 0.01$) from the corners of the previous frame. Corners with `status == 1` are the matches, and the provided `inlierMaskComputation` (fundamental matrix + RANSAC) selects the inliers. Only the inliers are tracked further. Every frame, `goodFeaturesToTrack` (`useHarrisDetector = true`, $k = 0.04$, quality level 0.01) tops up the set with new Harris corners, masked to stay at least the minimum distance away from the corners already tracked. `show()` draws the last 20 positions of every track in green and the RANSAC outliers in red (Fig. 12; videos in `video/d7_lk_tracking_*.mp4`). For the statistics, "Keypoints in Img 1" are the corners LK tries to track, "Keypoints in Img 2" the corners available in the current frame after re-detection, "Matches" counts one query per corner (as for the descriptor methods), and "Good matches" are corners with `status == 1`. The results are in the last column of Tables 3–4.



(a) `30fps_424x240`, frame 150. **(b)** `smooth-trajectory`, frame 150.

Figure 12: Harris + LK feature tracks (last 20 frames of every track, green; RANSAC outliers red).

The average number of corners stays below the maximum (561 of 1000 on the real data, 276 on the simulated data) because `goodFeaturesToTrack` only accepts corners whose Harris response exceeds 1 % of the strongest corner in the image.

Assumptions of the LK tracker.

1. **Brightness constancy:** a feature keeps the same intensity between frames, $I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$; linearising gives the optical flow constraint $I_x u + I_y v + I_t = 0$.
2. **Small motion:** the displacement is small enough for that first-order Taylor expansion to hold. The image pyramid extends this to larger motions, coarse to fine.
3. **Spatial coherence:** all pixels in the window around a feature share the same translational flow, which gives an over-determined system solved by least squares. The system is well conditioned only if the 2×2 structure tensor $\sum \nabla I \nabla I^\top$ has two large eigenvalues, and that is exactly what the Harris detector selects; hence the pairing of Harris corners with LK.

Comparison with the descriptor-based methods. Harris+LK has the highest inlier ratio on both datasets (99.7 % and 95.5 %), is among the fastest (3.0 and 5.9 ms per pair), and its inlier count (552 and 255) is comparable to ORB and FAST+BRIEF. It is fast because it computes no descriptors, searches locally and reuses the corners of the previous frame. It is reliable because

LK searches for each feature only around its previous position, so it cannot produce the distant ambiguous matches that a global descriptor search can (between repeated gates or floor tiles, for example), and tracking only inliers keeps bad points from propagating. In the per-frame plots (Fig. 9) LK has the steadiest curve, while the descriptor methods dip at fast rotations. The distributions in Fig. 13 show the same: the 5th percentile of the LK inlier ratio is 98.7 % on the real and 88.2 % on the simulated data, against 86.4 % and 75.6 % for SIFT. Its strength comes from its assumptions: LK needs small inter-frame motion and constant brightness, and long tracks accumulate drift. Matching unrelated views, as in the box pair of Deliverable 6.a, loop closure or re-localisation, remains the job of descriptors.

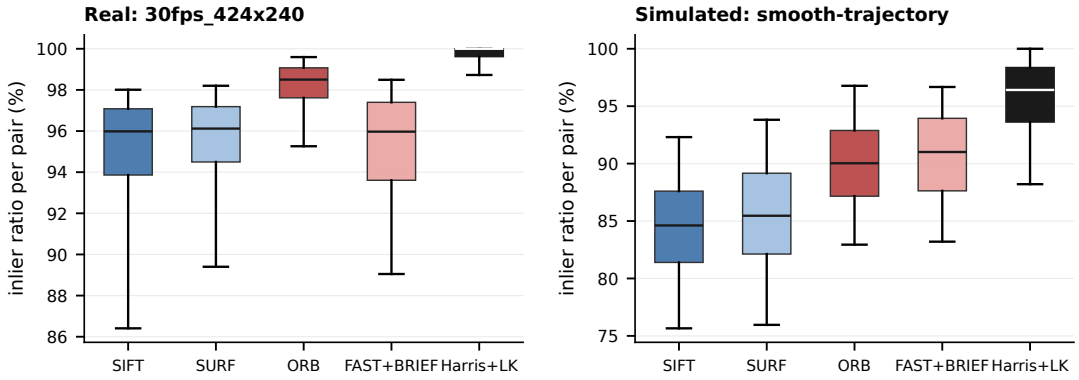


Figure 13: Distribution of the per-frame inlier ratio for the configurations of Tables 3–4 (1578–1583 pairs per method on the real, 1384–1385 on the simulated data). Boxes: quartiles and median; whiskers: 5th and 95th percentiles.

2.8 Deliverable 8 (optional): Farneback dense optical flow

The node in `self_flow.cpp` computes the dense flow between consecutive grayscale frames with Farneback's polynomial-expansion method [10] (`calcOpticalFlowFarneback`; pyramid scale 0.5, 3 levels, 15×15 window, 3 iterations, polynomial neighbourhood 5, $\sigma = 1.2$). It shows, side by side, the frame with the flow vectors on a 16-pixel grid and a colour-coded flow field (hue = direction, brightness = magnitude, saturated at 10 px). We ran it on both datasets (`roslaunch lab_5 self_flow.launch`).

Figure 14 shows one frame of each dataset; `video/d8_farneback_flow_*.mp4` contains the full sequences.

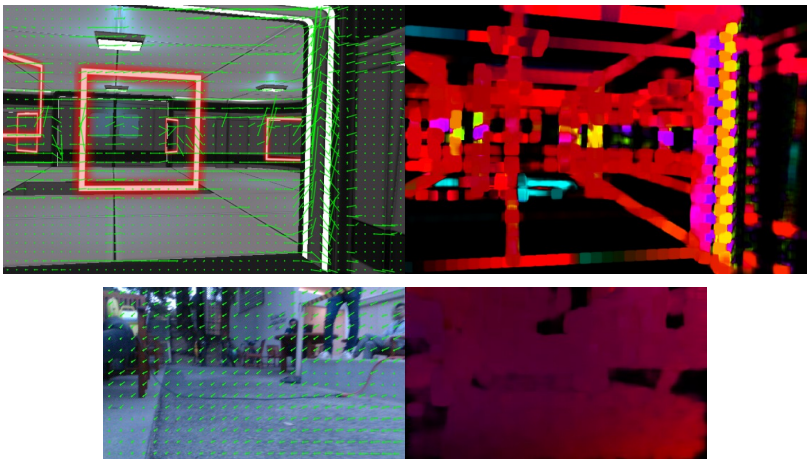


Figure 14: Farneback dense optical flow. Left: frame with flow vectors (2× scaled); right: colour-coded flow. Top: simulated dataset, bottom: real dataset.

During the rotation in the simulated frame, the flow is nearly uniform (one colour) on the far walls and much larger on the nearby door frame on the right, the parallax expected from the closer surface. Textureless regions (uniform walls, floor) get near-zero, unreliable flow: where the image has no gradient, its motion cannot be estimated.

Individual responsibility

Within the team I was responsible for the Harris corner detection, the Lucas–Kanade optical-flow tracker and the ROS video processing (Section 2.7), and I implemented the core of the Farneback dense optical flow (Section 2.8).

References

[1] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004. doi: 10.1023/b:visi.0000029664.99615.94. URL `https://doi.org/10.1023/b:visi.0000029664.99615.94`.

[2] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. SURF: Speeded up robust features. In *European Conference on Computer Vision (ECCV)*, pages 404–417, 2006.

[3] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. ORB: An efficient alternative to SIFT or SURF. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2564–2571, 2011.

[4] Edward Rosten and Tom Drummond. Machine learning for high-speed corner detection. In *European Conference on Computer Vision (ECCV)*, pages 430–443, 2006.

[5] Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua. BRIEF: Binary robust independent elementary features. In *European Conference on Computer Vision (ECCV)*, pages 778–792, 2010.

[6] Marius Muja and David G. Lowe. Fast approximate nearest neighbors with automatic algorithm configuration. In *International Conference on Computer Vision Theory and Applications (VISAPP)*, pages 331–340, 2009.

[7] Raul Mur-Artal, J. M. M. Montiel, and Juan D. Tardos. ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5):1147–1163, 2015. ISSN 1941-0468. doi: 10.1109/tro.2015.2463671. URL `https://doi.org/10.1109/tro.2015.2463671`.

[8] Chris Harris and Mike Stephens. A combined corner and edge detector. In *Alvey Vision Conference*, pages 147–151, 1988.

[9] Bruce D. Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 674–679, 1981.

[10] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Scandinavian Conference on Image Analysis (SCIA)*, pages 363–370, 2003.